



CODE SPORT

This month's column looks at how software bloat has become increasingly common and is taking a toll on performance.

For the last couple of months, we have been discussing dynamic languages such as JavaScript, and how they differ from traditional statically compiled languages like C or C++. One of the main differences of programming in the Web-centric world of Java, JavaScript, etc, is the increasing use of frameworks and libraries. While this facilitates rapid development, one of the concerns levelled against framework-based development is the excessive software bloat it inadvertently brings in. In this week's column, we take a look at this phenomenon.

Myhrvold's Laws

All of us have heard of Moore's Law, which states that the number of transistors on a chip doubles approximately every 18 months. This in turn implies that hardware becomes twice as fast and powerful every two years. However, not many of us are familiar with Myhrvold's Laws, which deal with the evolution of software. Nathan Myhrvold, a former CTO of Microsoft, proposed the following:

1. Software is a gas. This essentially means that software, like gas, expands to fill or utilise its container (namely, the hardware). No matter how fast the hardware you get, you can be sure that the next version of your favourite software will use it all up and will demand faster hardware. For instance, the number of lines of code for the Windows Vista operating system in 2008 was almost 150 times as large as that of Windows 3.1 in 1992.
2. Software grows until it becomes limited by Moore's Law. Essentially, as you get faster hardware, software developers keep adding functionality to take advantage of the new hardware, till the Moore's law induced improvements in the hardware are stretched to their limit.
3. It is the growth of software that makes Moore's Law possible (by making it economically

viable). This follows from point (2)—software developers keep adding functionality till the existing hardware cannot meet the performance requirements; the software vendors then start clamouring for faster hardware, which in turn provides economic motivation for hardware vendors to double up hardware speed.

4. It is impossible to have enough of software. This essentially means that there will always be demand for new functionality, new algorithms and new software paradigms.

In essence, Myhrvold's Laws portray the ever-growing complexity and size of software in terms of greater functionality and newer algorithms, which in turn typically means more code. This is all the more so in the Web-centric decentralised software world of today, wherein programs are no longer monolithic, but consist of modules imported from multiple frameworks and libraries, with a small amount of glue code. This in turn means that the software application you are using is not written by a single developer, but multiple developers who are probably not aware of the exact use to which their library or framework will be put. This results in software libraries with generic functionality, usable with multiple types of data. Besides, there is great emphasis on software re-use, so that the pressure is on the developer to write minimal custom code and use library or framework code as much as possible.

Software bloat

While the ability to re-use framework or library code improves programmer productivity, it also has an impact on performance. A framework needs to be generic enough such that its API can be used in multiple use-case scenarios. The flip-side of this is that a generic routine may not be the most efficient piece of code for the specific use-case scenario you have in your application. Let us consider a simple example.